

Building a Clerk-Class Authentication Platform: Product & Technical Requirements Document

Research dates: September 13, 2026. **Primary sources:** Clerk official docs (clerk.com/docs), Clerk Backend API reference, Clerk changelog, Clerk engineering blog, Slix docs, and OAuth/OIDC/WebAuthn/SCIM specifications, cross-checked against developer-community sources. **Evidence labels used throughout:** **[V]** Verified Clerk functionality; **[I]** Inferred from documentation; **[S]** Industry-standard implementation; **[R]** Architectural recommendation; **[U]** Unknown/undocumented.

TL;DR

- **Clerk is a full identity platform, not just login.** Its defensible moats are (1) a hybrid stateful/stateless session model with **60-second session-token JWTs auto-refreshed every ~50 seconds**, (2) best-in-class prebuilt React/Next.js components and developer experience, and (3) a bundled B2B stack (Organizations, RBAC, SAML/OIDC/EASIE SSO, SCIM directory sync GA in 2026, Billing). Replicating it requires ~18 backend services around a Postgres + Redis core, with security engineering as the hardest, highest-risk work.
 - **A credible MVP is buildable by a ~7-person team in ~4–6 months** covering email/password + passwordless + social login, sessions/JWTs with JWKS, user management, one prebuilt sign-in component, a Node/JS SDK, and webhooks. The multi-month, specialist-heavy work is enterprise SSO/SCIM, passkeys/WebAuthn, bot/risk ML, and multi-region scale — buy or defer these.
 - **Recommended build strategy: buy the commodity, high-liability pieces, build the differentiators.** Clerk's own architecture proves this — it uses Slix for webhooks and Cloudflare Turnstile for bot protection. Differentiate against Clerk on price at scale, self-hosting, and EU data residency (gaps Clerk currently has).
-

Key Findings

1. **The session/token engine is the crown jewel and the hardest thing to replicate.**
Clerk decouples token expiry (60s) from session lifetime (days/weeks) using two tokens, achieving stateless verification performance *and* near-instant revocation. Getting this correct and revocable is the single biggest engineering risk.
2. **Security is verified and specific:** bcrypt hashing, HaveIBeenPwned breach detection, NIST password rules, Cloudflare Turnstile bot protection, account lockout

(default 10 attempts / 1-hour cooldown as of July 6, 2026), RS256 JWTs verified via JWKS.

3. **The B2B stack is now enterprise-complete:** SAML/OIDC/EASIE SSO plus SCIM Directory Sync went GA April 16, 2026 (group/attribute mapping May 21, 2026), closing the gap competitors used to cite.
4. **Pricing changed materially in 2026:** the free tier jumped from 10,000 to **50,000 Monthly Retained Users on February 5, 2026**; Pro is \$25/mo + \$0.02/MRU overage.
5. **Clerk's weaknesses are a competitor's opportunities:** cost at scale, no self-hosting, no EU data residency, and middleware footguns.

Details

PART 1 — Complete Clerk Service Inventory

Clerk is a multi-product identity platform, not a single auth widget. Verified surface [V]:

Domain	Capabilities
Authentication	Email/password, email/phone verification codes, magic links, SMS codes, passwordless, social OAuth (28 providers), passkeys/WebAuthn, MFA (TOTP, SMS, backup codes), Web3/Metamask sign-in
User management	Profiles, multiple emails/phones, profile images, public/private/unsafe metadata, custom attributes, ban/lock, impersonation, invitations, user export, CRUD, search
Sessions	Hybrid session model, client token + session token, multi-device, active device monitoring, revocation, custom session lifetimes (paid)
Tokens	Session-token JWTs (v2 format), JWT templates, custom claims, JWKS, RS256 signing
Organizations (B2B)	Orgs, memberships, default + up to 10 custom roles, permissions, invitations, org switching, org metadata, verified domains, org-level billing
Enterprise	SAML 2.0, OIDC, EASIE SSO (Google/Microsoft), SCIM 2.0 directory sync, JIT provisioning, group→role mapping, domain-based enrollment
Developer platform	Backend API (BAPI), Frontend API (FAPI), webhooks (Svix), JWT templates, API keys, CLI, agent "skills"

Domain	Capabilities
Frontend	Prebuilt components, Account Portal (hosted), Clerk Elements (headless), custom flows
Security	bcrypt hashing, HavelBeenPwned breach detection, Cloudflare Turnstile bot protection, account lockout/brute-force protection, IP rate limiting, credential-stuffing/"Client Trust" step-up
Billing (product)	Subscriptions for B2C/B2B, plans, payments, billing webhooks, <PricingTable/>
AI/agents	API keys product, agent CLI/skills, OAuth for AI agents (CIMD/Dynamic Client Registration)
SDKs	Frontend: Next.js, React, React Router, Remix, Expo, Astro, TanStack Start, Vue, Nuxt, JS, Chrome Extension, iOS, Android, Flutter. Backend: Node, Go, Python, Ruby, Java, PHP, C#

PART 2 — Authentication

Architecture context [V]: Clerk provisions a dedicated **Frontend API (FAPI)** per app at `https://<slug>.clerk.accounts.dev` (dev) or a CNAME'd subdomain (prod). FAPI handles per-user flows (sign-up, sign-in, session retrieval). The **Backend API (BAPI)** at `api.clerk.com` handles admin actions (list users, ban, impersonate) authenticated by a secret key. The Publishable Key is the base64-encoded FAPI URL, prefixed `pk_test_ / pk_live_`.

2.1 Email/Password — bcrypt hashing [V]; imports support argon2/scrypt/pbkdf2 with rehash-on-login to bcrypt. NIST character rules + HavelBeenPwned breach check. Data: `password_credential1`. FAPI `POST /v1/client/sign_ups , /sign_ins , attempt_first_factor`. **Complexity:** Low–Medium. **MVP:** P0.

2.2 Passwordless / codes / magic links — OTP (email/SMS) and magic links; single-use, short TTL [S]. FAPI limits: create sign-in/sign-up **5 req/10s**, attempt **3 req/10s** [V]. **Complexity:** Medium. **MVP:** P1.

2.3 Social login (28 providers) [V] — see Part 6.

2.4 Passkeys/WebAuthn [V] — FIDO2 credentials backed by device biometrics; paid plans (free in dev); "a passkey is itself a multi-factor credential." Data: `passkey` (`credential_id`, `public_key`, `sign_count`, `transports`). **Complexity:** High. **MVP:** P2.

2.5 MFA [V] — SMS code, authenticator app (TOTP), and backup codes; enforced automatically at sign-in. Clerk provides **no end-user MFA reset flow** — developers build recovery. The `fva` claim (factor verification age) enables step-up. **Complexity:** Medium–High. **MVP:** P1 (TOTP+backup).

2.6 Recovery / reset — verified email/phone code; MFA recovery via backup codes.
MVP: P1.

2.7 Sessions / revocation / step-up — see Parts 4–5; step-up via `useReverification()`.

2.8 Bot / brute-force / rate limiting [V] — Cloudflare Turnstile "Smart" mode (invisible for 99.9% of users; the older "Invisible" type is deprecated); account lockout (min 5, default 10 attempts, configurable duration incl. indefinite, admin unlock); ML-based bot/multi-account detection + "Client Trust" second-factor-for-new-devices; IP-based FAPI rate limits. **Complexity:** ML risk engine = Very High (buy/defer); CAPTCHA integration = Low. **MVP:** P0 (CAPTCHA + lockout + rate limit); P3 (ML risk).

PART 3 — User Management

Verified [V]:

- **Metadata (3 types):** `public` (read frontend+backend, write backend only), `private` (backend only), `unsafe` (read/write frontend). Total limit **8KB** (<1.2KB recommended if in session token). Server-side changes don't appear in the token until next refresh — force a refresh to avoid race conditions.
- **Impersonation:** dashboard one-click or actor tokens; `act` claim identifies impersonator; **5 free impersonations/month**.
- **Invitations:** BAPI `POST /v1/invitations` (100/hr; bulk 25/hr); public metadata flows into the user on sign-up.
- **Ban/lock/delete:** `DELETE /v1/users/{id}` destroys user, all sessions, memberships.
- **External ID / linking:** store your own FK as `externalId`; multiple emails/phones/OAuth accounts per user; social sign-up/sign-in are equivalent.
- **Export:** self-serve dashboard export on every plan (no lock-in). **MVP:** P0 for CRUD/profile/metadata; P1 invitations/impersonation; P2 account merging.

PART 4 — Session Management Architecture (the core insight) [V]

On sign-in, Clerk:

1. Creates a **session record** in DB (stateful — enables instant revocation).
2. Issues a **client token** (`__client` cookie, `HttpOnly`, `SameSite=Lax`, on the FAPI domain, long-lived, containing `id` + a rotating anti-fixation `rotating_token`). Production only; source of truth for auth state.
3. Issues a **session token** (short-lived JWT, **60s expiry**, containing user/session claims, on the app domain).

The client-side SDK refreshes the session token on a **~50-second interval** (10s latency buffer) via `POST /v1/client/sessions/{id}/tokens`. This decouples token expiry

(60s) from session lifetime (days/weeks): stateless verification performance + near-instant revocation.

Sign-up + email verification:

```
sequenceDiagram
    participant U as User/Browser
    participant F as FAPI
    participant DB as DB
    participant E as Email svc
    U->>F: POST /sign_ups (email,password)
    F->>DB: create user(pending), verification
    F->>E: send code
    E-->>U: code
    U->>F: POST attempt_verification(code)
    F->>DB: mark verified, create session
    F-->>U: Set-Cookie __client; session active
```

Sign-in + MFA + token issuance:

```
sequenceDiagram
    participant U as Browser
    participant F as FAPI
    participant DB as DB
    U->>F: POST /sign_ins (identifier)
    F->>DB: verify first factor
    F-->>U: needs_second_factor
    U->>F: attempt_second_factor(TOTP)
    F->>DB: verify + create session
    F-->>U: Set-Cookie __client
    U->>F: POST /sessions/{id}/tokens
    F-->>U: session JWT (60s)
```

API request + refresh + expiration:

```
sequenceDiagram
    participant U as Browser
    participant A as App backend
    participant J as JWKS cache
    loop every ~50s
        U->>U: refresh session token
    end
    U->>A: request + Bearer session JWT
    A->>J: get public key (kid)
    A->>A: verify sig, exp, nbf, iss, azp
    A-->>U: 200 / 401 if expired
```

Logout / revocation:

```
sequenceDiagram
    participant U as Browser
    participant F as FAPI
    participant DB as DB
    U->>F: sign out
    F->>DB: set session revoked
    F-->>U: clear __client
    Note over DB: next token refresh fails; session ends within ≤60s
```

Server vs client auth [V]: `clerkMiddleware()` populates auth state but does **not** protect routes by default (opt-in). The real security boundary is the server-side `auth()` check close to the resource; client `useAuth()` / `useUser()` are for presentation only.
MVP: P0. **Complexity:** Very High.

PART 5 – Token System [V]

Session token (JWT v2) default claims: `azp` (origin), `exp`, `fva` (factor verification age), `iat`, `iss` (FAPI URL), `jti`, `nbfi`, `sid`, `sub` (user id), `v` (version=2), `pla` (active plan), `fea` (features), `sts` (session status). Optional `o` claim (org: `id`, `sig`, `rol`, `per`, `fpm` feature-permission bitmask) and `act` claim (impersonation). v1 deprecated April 14, 2025.

JWT templates: generate custom JWTs for third-party services; default claims always included; `sid` only in session-bound tokens; custom claims add latency.
`getToken({template})` client-side; BAPI `getToken(sessionId, template)`.

Signing / JWKS: tokens signed with the instance **private key** and verified with the **public key**; header includes `alg`, `typ`, `kid`; algorithm is RS256 (asymmetric). JWKS published for verification; `GET /v1/jwks` has **no rate limit**. Networkless verification by passing `jwtKey`. **Key rotation [R/S]:** publish new key with new `kid`, sign new tokens, keep old public key until outstanding tokens expire (60s makes this trivial), then remove.

Verification (documented): `clerkClient.authenticateRequest(req, {jwtKey, authorizedParties})` or manual `jwt.verify(token, publicKey)` reading `__session` cookie (same-origin) or `Authorization` header (cross-origin); validate `exp`, signature, `azp` / authorized parties. **MVP:** P0. **Complexity:** High.

PART 6 – Social Login & Federated Identity [V]

- **Providers:** 28 built-in social connections (Google, Microsoft, GitHub, Apple, Facebook, LINE, Linear, HuggingFace, Atlassian, etc.). Dev uses Clerk's shared preconfigured OAuth credentials; prod requires custom credentials per provider.
- **Flow [V/S]:** standard OAuth 2.0 authorization-code + PKCE; Clerk manages authorization URL, `state`, callback, token exchange, identity mapping. Sign-up/sign-in are equivalent.

- **Native flows:** Sign in with Google via Android Credential Manager / iOS native; ID token sent to Clerk and verified.
- **Access tokens:** `getUserOAuthAccessToken()` server-side to call provider APIs; custom scopes supported.
- **Security:** nOAuth mitigation for Microsoft Entra (stricter verified-email checks + optional `xms_edov` claim). [Clerk MVP](#): P1 (Google + GitHub). **Complexity:** Medium.

PART 7 — Organizations & B2B Auth

```
erDiagram
    USER ||--o{ ORG_MEMBERSHIP : has
    ORGANIZATION ||--o{ ORG_MEMBERSHIP : contains
    ORGANIZATION ||--o{ ORG_INVITATION : issues
    ORGANIZATION ||--o{ ORG_DOMAIN : verifies
    ROLE ||--o{ ROLE_PERMISSION : grants
    PERMISSION ||--o{ ROLE_PERMISSION : in
    ORG_MEMBERSHIP }o--|| ROLE : assigned
```

Verified [V]:

- Default roles `org:admin`, `org:member`; up to **10 custom roles per instance** (contact support beyond). Custom roles require a paid plan (Enhanced B2B add-on) in prod, free in dev.
- **9 System Permissions** (`org:sys_profile:manage`, `org:sys_memberships:read/manage`, `org:sys_domains:read/manage`, `org:sys_billing:read/manage`, `org:sys_entconns:manage`, `org:sys_profile:delete`); custom permissions have no documented numeric cap.
- **Limits:** users can create up to **100 orgs** by default; **unlimited** org memberships; **5 members/org** default, **20** max without B2B add-on, **unlimited** with it (`max_allowed_memberships=0` = unlimited). Free plan: 50 monthly retained orgs (dev) / 100 (prod).
- **Two RBAC approaches:** Organizations (B2B) or user `publicMetadata` roles (B2C). Org context surfaced in the JWT `o` claim; switching via `<OrganizationSwitcher/>`; verified domains enable auto-join. Roles/permissions fully manageable via BAPI since Nov 24, 2025. **MVP**: P1 (orgs + default roles); P2 (custom roles). **Complexity:** High.

PART 8 — Enterprise (SAML/OIDC/EASIE/SCIM) [V]

- **SAML 2.0** (SP- and IdP-initiated), **OIDC**, first-class Entra ID, Google Workspace, Okta + any SAML IdP.
- **EASIE SSO:** Clerk's open spec (easie.dev) delivering enterprise SSO over OIDC via Google/Microsoft, identified by email domain, with **zero per-customer credential**

exchange; [Clerk](#) a deprovisioning check (≤ 10 -min delay) runs before issuing new session tokens. [Clerk](#) Clerk is the only platform publicly implementing EASIE as of June 2026. [Clerk](#)

- **SCIM 2.0 Directory Sync:** GA **April 16, 2026**; group $\rightarrow$ role + custom attribute mapping GA **May 21, 2026**. Clerk is the SCIM **service provider**; deprovisioning **immediately revokes all sessions**. Documented for Okta and Entra ID (Google Workspace is not a SCIM source); requires an existing SAML/OIDC connection; included free with each enterprise connection.
- **JIT provisioning:** first SAML sign-in creates the user, adds them to the org owning the verified domain, assigns the default role.
- **Self-serve SSO:** customers configure their own SAML (June 26, 2026) / custom OIDC (July 31, 2026).

```
sequenceDiagram
    participant U as User
    participant App
    participant C as Clerk
    participant IdP
    U->>App: enter work email
    App->>C: lookup domain $\rightarrow$ connection
    C-->>U: redirect to IdP (SAML/OIDC)
    U->>IdP: authenticate
    IdP-->>C: assertion/id_token
    C->>C: JIT provision + map attrs/groups $\rightarrow$ roles
    C-->>App: session
```

MVP: P3 / Enterprise phase. **Complexity:** Very High. **Recommendation:** buy WorkOS or build on a SAML library; SCIM last.

PART 9 – Webhooks & Event System [V]

- Delivered via **Svix** (Standard Webhooks protocol): headers `svix-id / svix-timestamp / svix-signature`; **HMAC-SHA256** over the raw body; secret `whsec_...`.
- **At-least-once, unordered** delivery; retries with exponential backoff up to **3 days**; dashboard replay. Verify with the raw body, return 200 fast, upsert on object `id` for idempotency.
- **Event catalog** (~40+ events; the full enumerated list is Dashboard-only):
`user.created/updated/deleted`;
`session.created/ended/removed/revoked/pending`;
`organization.created/updated/deleted`;
`organizationMembership.created/updated/deleted`;
`organizationInvitation.created/accepted/revoked`;
`organizationDomain.created/updated/deleted`; `role.*`, `permission.*`;
`email.created`, `sms.created`; `waitlistEntry.created/updated`; `dedicated`

billing events. **No dedicated SAML/SCIM webhook types** — SSO surfaces via `user.* / organizationMembership.*` [U].

- **To build equivalent [R]:** queue + delivery workers + HMAC signing + retry/backoff + dead-letter + replay. **Recommendation:** buy Svix. **MVP:** P1. **Complexity:** Medium (High if self-built).

PART 10 — Frontend Components [V]

Components: `<SignIn/>`, `<SignUp/>`, `<GoogleOneTap/>`, `<OAuthConsent/>`, `<Waitlist/>`, `<UserButton/>`, `<UserAvatar/>`, `<UserProfile/>`, `<CreateOrganization/>`, `<OrganizationProfile/>`, `<OrganizationSwitcher/>`, `<OrganizationList/>`, `<InviteMembersButton/>`, `<PricingTable/>`, `<CheckoutButton/>`; Clerk control components `<SignedIn/>`, `<SignedOut/>`, `<Show/>`, `<Protect/>`, `<ClerkLoaded/>`, buttons.

- **Customization:** `appearance` prop (variables, elements/CSS classes, layout, options, prebuilt themes incl. shadcn), `localization` prop, custom pages in `<UserProfile/>`. Clerk
- **Headless:** Clerk Elements + custom flows. **Hosted:** Account Portal. Branding removable on paid plans. Clerk MVP: P0 (`<SignIn/>`, `<SignUp/>`, `<UserButton/>`); P1 org components. **Complexity:** Very High to match polish/a11y (a key differentiator).

PART 11 — SDKs [V]

Next.js hooks: `useUser`, `useClerk`, `useAuth`, `useSignIn`, `useSignUp`, `useSession`, `useSessionList`, `useOrganization`, `useOrganizationList`, `useReverification`, `useOAuthConsent` + billing hooks; `clerkMiddleware()` + `createRouteMatcher`; server `auth()`; `<ClerkProvider>`.

SDK	Client Auth	Server Auth	UI	Middleware	Webhooks	Orgs	Mobile
Next.js	✓	✓	✓	✓	✓ (verify)	✓	—
React	✓	—	✓	—	—	✓	—
Remix / React Router / Astro / TanStack	✓	✓	✓	✓	✓	✓	—
Expo / React Native	✓	—	✓ (native)	—	—	✓	✓

SDK	Client Auth	Server Auth	UI	Middleware	Webhooks	Orgs	Mobile
iOS / Android / Flutter	✓	—	✓	—	—	✓	✓
Node / Express / Fastify / Hono / Koa / Elysia	—	✓	—	✓	✓	✓	—
Go / Python / Ruby / Java / PHP / C#	—	✓	—	✓	✓	✓	—

Expo uses `expo-secure-store` (iOS Keychain / Android Keystore) via `@clerk/expo/token-cache`. **MVP:** P0 JS/Node + Next.js; P1 React/Express; P2 mobile.

PART 12 — Backend API [V]

BAPI is a Bearer-secret-key REST API at `api.clerk.com/v1` (~109 endpoints, OpenAPI available). Groups: Users, Sessions, Organizations, Memberships, Invitations, Phone/Email templates, SAML connections, Sign-in tokens (one-time `ticket` strategy), Redirect URLs, Domains, JWKS, JWT templates, Testing tokens, Roles/Permissions, OAuth apps, Webhooks.

Rate limits [V]: prod **1000 req/10s**, dev **100 req/10s**; per-user/org metadata updates **10/10s**; invitations **100/hr** (bulk 25/hr); org invitations **250/hr** (bulk 50/hr); JWKS unlimited. 429 returns `Retry-After`. **MVP:** P0 users/sessions; P1 orgs/invitations.

PART 13 — Developer Dashboard [V]

Applications; **dev vs prod environments** (dev = shared OAuth creds, all features free; prod = custom creds); API keys (`pk_/sk_`, `_test_/_live_`); social/email/SMS providers; auth/MFA/session settings; SSO connections; **Webhooks** (Event Catalog, signing secret, Message Attempts/replay); **JWT templates**; **Domains** (FAPI URL, CNAME); allowed origins; redirect URLs; **Attack protection** (bot, lockout); appearance/branding; localization; **Updates** page (version pinning); user management (impersonate/ban/delete); Directory Sync tab per connection. Backend systems required [R]: config service, secrets/KMS, per-tenant settings store, audit log, metrics/log pipeline. **MVP:** P0 minimal (keys, providers, users).

PART 14 — Security Architecture (STRIDE-aware)

Threat	Attack	Protection [V]/[R]	Failure mode
Password theft	DB leak	bcrypt salted; rehash imports; HavelBeenPwned check [V]	weak cost → cracking; use bcrypt≥12 / Argon2id [R]
Session hijack	stolen cookie	60s token + rotating client token; HttpOnly/SameSite/HTTPS; separate cookies per domain/subdomain [V]	XSS if not HttpOnly
Session fixation	forced id	rotating_token rotates each sign-in [V]	—
Token forgery	alg:none /RS→HS	RS256 + explicit alg allowlist [R/S]	accepting attacker kid
CSRF	cross-site POST	SameSite + origin/azp check [V/S]	lax config
Brute force	guessing	lockout (10/1hr) + rate limits [V]	too-high limit
Credential stuffing	breached lists	ML bot detection + Client Trust step-up [V]	—
Bot sign-ups	fake accounts	Cloudflare Turnstile Smart [V]	—
Account enumeration	probing	generic responses [R/S]	leaking "user exists"
OAuth attacks	nOAuth spoofing	verified-email enforcement, xms_edov [V]	trusting unverified email
Replay	reused OTP/token	single-use codes, jti, short TTL [S]	—
Tenant isolation	cross-tenant	per-instance keys, row scoping [I/R]	missing tenant filter

Threat	Attack	Protection [V]/[R]	Failure mode
Secret exposure	leaked keys	secret key server-only; client token on FAPI domain [V]	committing sk_

Clerk is **SOC 2 Type 2 and CCPA compliant**, runs third-party audits/pen tests, and is **US-based with no EU data residency currently**. Password hashing, key management, and OAuth/SAML security require specialized expertise — do not improvise.

PART 15 — Data Model (proposed) [R]

```

erDiagram
    USER ||--o{ EMAIL_ADDRESS : has
    USER ||--o{ PHONE_NUMBER : has
    USER ||--o{ EXTERNAL_ACCOUNT : has
    USER ||--o{ PASSWORD_CREDENTIAL : has
    USER ||--o{ PASSKEY : has
    USER ||--o{ MFA_FACTOR : has
    USER ||--o{ BACKUP_CODE : has
    USER ||--o{ SESSION : has
    SESSION ||--o{ SESSION_TOKEN : issues
    SESSION }o--o{ DEVICE : on
    USER ||--o{ ORG_MEMBERSHIP : has
    ORGANIZATION ||--o{ ORG_MEMBERSHIP : contains
    ORGANIZATION ||--o{ INVITATION : issues
    ORGANIZATION ||--o{ DOMAIN : verifies
    ROLE ||--o{ ROLE_PERMISSION : grants
    PERMISSION ||--o{ ROLE_PERMISSION : in
    ORG_MEMBERSHIP }o--|| ROLE : has
    APPLICATION ||--o{ USER : scopes
    APPLICATION ||--o{ WEBHOOK_ENDPOINT : has
    WEBHOOK_ENDPOINT ||--o{ WEBHOOK_EVENT : delivers
    APPLICATION ||--o{ SSO_CONNECTION : has
    USER ||--o{ VERIFICATION : has
    USER ||--o{ SECURITY_EVENT : logs

```

Notes [R]: opaque prefixed PKs (user_, sess_, org_); tenant application_id FK on every row; **encrypted (KMS envelope)**: password_credential.hash, mfa_factor.secret, backup_code.hash, OAuth refresh tokens, SSO private keys. Metadata as JSONB (8KB cap). Indexes on (application_id, email), (application_id, session_status), (org_id, user_id). Session lifecycle active→ended|revoked|expired.

PART 16 — System Architecture

```

graph TD
  CDN[CDN/Edge] --> GW[API Gateway/FAPI+BAPI]
  GW --> AUTH[Auth svc]
  GW --> USER[User svc]
  GW --> SESS[Session svc]
  GW --> TOK[Token/JWKS svc]
  GW --> OAUTH[OAuth svc]
  GW --> SSO[SSO/SCIM svc]
  GW --> ORG[Org/Authz svc]
  AUTH --> VERIF[Verification svc]
  VERIF --> EMAIL[Email svc]
  VERIF --> SMS[SMS svc]
  AUTH --> RISK[Risk/Bot svc]
  GW --> RL[Rate-limit svc/Redis]
  SESS --> REDIS[(Redis)]
  USER --> PG[(Postgres)]
  ORG --> PG
  SESS --> PG
  subgraph Async
    MQ[Queue] --> WHK[Webhook svc/Svix]
    MQ --> AUD[Audit svc]
    MQ --> WORK[Workers]
  end
  end
  USER --> MQ
  TOK --> KMS[KMS/Secrets]
  DASH[Admin/Dev Dashboard] --> GW

```

FAPI (per-user, public) and BAPI (admin, secret-key) are distinct gateways [V]. Token verification is networkless at the app edge via cached JWKS [V]. **MVP [R]**: start as a modular monolith (auth+user+session+token+org in one deployable) with Postgres+Redis; split services only at scale.

PART 17 — Infrastructure [R]

Component	Choice	Consistency
Compute	Containers (ECS/EKS) or serverless edge for verify	—
Primary DB	PostgreSQL (Aurora/RDS)	strongly consistent
Cache/sessions	Redis (revocation lists, rate limits)	eventually consistent
Queue	SQS/Kafka	at-least-once
Object storage	S3 (profile images, exports)	—

Component	Choice	Consistency
CDN	Cloudflare/CloudFront (JWKS, static, Turnstile)	edge-cached
Email	Resend/Postmark/SES	—
SMS	Twilio	—
KMS/HSM	Cloud KMS for signing-key envelope encryption	—
Secrets	Vault / cloud secrets manager	—
Observability	OpenTelemetry + Prometheus/Grafana + Datadog	—

Centralize signing keys, config, audit. Distribute JWKS (CDN, 99.99%) and token verification (stateless edge). Clerk's lack of EU residency is a differentiation opportunity.

PART 18 — Reliability & Scalability (1K → 100M users)

Scale	Key moves	Bottleneck
1K	Single Postgres + Redis, monolith	none
10K	Read replica, JWKS on CDN	email/SMS deliverability
100K	PgBouncer, Redis cluster	sign-in write throughput
1M	Read replicas, cache lookups, queue all webhooks/email	hot rows on popular orgs
10M	Partition by <code>application_id</code> ; multi-AZ; regional JWKS	token refresh QPS
100M	Multi-region, user partitioning, dedicated risk pipeline	token-refresh storm

The 60s-token/50s-refresh design trades refresh volume for revocation speed — at 100M users that's roughly **~2M refresh req/s** ($\text{users} \div 50$), the defining scaling cost. Mitigate with edge caching, adaptive refresh (only when active), and a configurable longer token TTL [R].

PART 19 — Observability & Operations

Emit structured logs, metrics, traces (OTel), audit logs, security/auth events, API logs, webhook delivery logs, failed-login tracking. **Proposed SLOs [R]:**

SLI	SLO
Auth (sign-in) availability	99.95%
Token issuance/refresh	99.99%, p99 < 150ms
Token verification (edge)	p99 < 10ms
BAPI availability	99.9%
Webhook delivery (eventual)	99.5% within 5 min
Email verification delivery	98% within 60s
SMS delivery	95% within 60s

Runbooks: revocation, key-compromise rotation, on-call for auth (highest severity – auth down = customers' apps down).

PART 20 — Billing & Business Model

Clerk's actual pricing (verified, 2026) [V]:

- **Free (Hobby):** 50,000 Monthly Retained Users (MRU) — "Clerk replaced its long-running 10,000-user free tier with the 50,000-MRU tier on February 5, 2026"; 100 monthly retained orgs; full user export on every plan.
- **Pro:** \$25/mo — removes branding, adds MFA + custom session lifetimes, **1 enterprise SSO connection included**; overage **\$0.02/MRU** (50,001–100,000 band) with volume discounts down to **\$0.012/MRU at 10M+**. "At 100,000 retained users you'll pay about **\$1,025/mo** on Pro."
- **Business:** \$300/mo — 10 dashboard seats, 30-day log retention, SOC 2 report, priority support.
- **Enterprise:** custom, SLAs, HIPAA, dedicated support.
- **Add-ons:** Enhanced B2B SaaS (custom roles, unlimited org members); additional SSO ~\$75/connection.
- **Unit = MRU**, deliberately narrower than MAU.

Recommended pricing for our platform [R]: Free to 50K MAU to match market; usage-based overage ~\$0.015–0.02/MAU with volume tiers; SSO priced per connection but cheaper than WorkOS (see Part 22); bundle SCIM free with SSO; flat B2B add-on for unlimited org members; offer **EU data residency** and **self-hosting** as paid differentiators Clerk lacks.

PART 21 — Developer Experience

Strengths [V/community]: time-to-first-auth under 10 minutes (drop-in Next.js component); best-in-class prebuilt components; all features free in dev; excellent docs/CLI/agent skills; native org support (Supabase lacks it).

Weaknesses / pain points: cost at scale (#1 complaint; teams migrate ~50K+ MAU); vendor-lock-in perception (mitigated by export); no EU residency, no self-hosting; middleware footguns (`clerkMiddleware` doesn't protect by default; 404/root-layout mismatch; client-vs-server import confusion); enterprise features historically felt "bolted on" (SCIM only GA in 2026); 4KB cookie / 1.2KB custom-claim limits surprise teams.

PART 22 — Competitive Analysis

Capability	Clerk	Auth0	Firebase	Supabase	Cognito	WorkOS	Sty
DX / UI components	★★★★★	★★★	★★	★★★	★	★★★	★★
Social login	✓28	✓	✓	✓	✓	✓	✓
MFA/passkeys	✓	✓	partial	✓	✓	✓	✓
Organizations	✓ native	✓	✗	✗	limited	✓	✓
SSO (SAML/OIDC)	✓+EASIE	✓	✗	via 3P	✓	✓✓	✓
SCIM	✓ GA 2026	✓	✗	✗	✗	✓✓	✓
Webhooks	✓ Svix	✓	✓	✓	✓	✓	✓
Self-host	✗	✗	✗	✓	✗	✗	✗
Free tier	50K MRU	25K MAU	50K MAU	50K MAU	10K MAU	1M MAU	101

Verified pricing anchors [V]: WorkOS AuthKit is **free to 1,000,000 MAU**, then **\$2,500 per additional million**, with SSO at **\$125/connection for the first 15**, then **\$100 (16–30)**, **\$80 (31–50)**, **\$65 (51–100)**, **\$50 (101–200)**. Auth0's free "Identity platform" tier is **25,000 MAU** (raised from 7,500 — "the largest free-tier expansion since the Okta acquisition"), including 1 enterprise connection, 5 organizations, and 1 custom domain. Supabase Auth includes **50,000 MAU free (100,000 on paid)** with overage of exactly **\$0.00325/MAU** (SSO MAUs bill separately at \$0.015/MAU, 50 free). Firebase Auth offers 50,000 free MAU.

Clerk's differentiation: DX + prebuilt components + bundled B2B (orgs+SSO+SCIM+billing in one SDK) + EASIE. **Weak spots:** price at scale (WorkOS/Supabase far cheaper at high volume), no self-host, no EU residency, enterprise depth vs WorkOS/Auth0.

PART 23 – Feature Prioritization

Feature	Priority	Complexity	Sec risk	Order
Email/password + verification	P0	M	High	1
Session/token engine + JWKS	P0	VH	Critical	2
User CRUD + metadata	P0	M	Med	3
SignIn/SignUp/UserButton	P0	VH	Med	4
JS/Node + Next.js SDK	P0	H	Med	5
Rate limit + lockout + CAPTCHA	P0	M	High	6
Social login (Google/GitHub)	P1	M	Med	7
Email-code passwordless	P1	M	Med	8
MFA (TOTP+backup)	P1	H	High	9
Webhooks	P1	M	Med	10
Organizations + default roles	P1	H	Med	11
Custom roles/permissions	P2	H	Med	12
Passkeys/WebAuthn	P2	H	Med	13
Enterprise SAML/OIDC	P3/Ent	VH	High	14
SCIM directory sync	P3/Ent	VH	High	15
Multi-region/EU residency	P3/Ent	VH	Med	16

PART 24 – Build vs Buy

Subsystem	Decision [R]	Rationale
Email	Buy (Resend/Postmark/SES)	deliverability
SMS	Buy (Twilio)	carrier relationships

Subsystem	Decision [R]	Rationale
CAPTCHA	Buy (Cloudflare Turnstile — Clerk uses this)	ML upkeep
Webhooks	Buy (Svix — Clerk uses this)	retries/signing/replay
OAuth client	OSS lib + orchestration	standard flows
SAML	OSS lib / Buy WorkOS	XML security landmines
SCIM	Build on spec / Buy	protocol edge cases
WebAuthn	OSS lib (SimpleWebAuthn)	don't hand-roll crypto
DB / Redis / Queue	Cloud managed	HA/backups
KMS / Secrets	Cloud KMS / Vault	key custody/compliance
Observability	Buy (Datadog) + OSS OTel	—
Session/token engine, components, orgs/RBAC, DX	BUILD	the differentiators

PART 25 — Product Requirements Document

Executive summary: Build a developer-first identity platform delivering authentication, user management, sessions, organizations/RBAC, and enterprise SSO/SCIM behind polished SDKs and prebuilt UI — matching Clerk's DX while differentiating on price-at-scale, self-hosting, and EU data residency.

Vision: "Auth a solo dev ships in 10 minutes and an enterprise trusts at 10M users."

Problem statement: DIY auth is high-risk; existing providers force a trade of DX (Clerk) vs cost (Supabase/Cognito) vs enterprise depth (Auth0/WorkOS) vs control (Keycloak).

Target users/personas: indie dev (speed), startup (speed+cost), B2C SaaS (scale+cost), B2B SaaS (orgs+SSO), enterprise admin (SCIM/compliance/residency).

Jobs to be done: "add secure login fast," "manage users/teams," "sell to enterprise without a rebuild," "stay compliant."

Principles [R]: secure-by-default; DX first; open standards (OIDC/SAML/SCIM/WebAuthn); no lock-in (export + self-host); transparent pricing.

Functional requirements: Parts 2–13. **Non-functional:** SLOs (Part 19); security (Part 14); scale (Part 18); compliance SOC 2 Type 2, GDPR, CCPA, HIPAA (enterprise).

PART 26 — User Stories (samples)

- As a **developer**, I want to add sign-in with one component so that I ship auth in an afternoon.
- As a **developer**, I want to verify a session token without a network call so that my API stays fast.
- As an **end user**, I want to sign in with Google or a passkey so that I avoid passwords.
- As an **end user**, I want to see and revoke my active devices so that I control my account.
- As an **org admin**, I want to invite teammates and assign roles so that access matches responsibility.
- As an **enterprise admin**, I want SAML SSO by email domain so that employees use corporate credentials.
- As an **enterprise admin**, I want SCIM deprovisioning to instantly revoke sessions so that offboarding is immediate.
- As a **developer**, I want webhooks for user/session/org events so that my DB stays in sync.

PART 27 — Acceptance Criteria (Given/When/Then, samples)

- **Sign-in happy path:** Given a verified user, When correct credentials are submitted, Then a session is created and a token issued within 150ms.
- **Brute force:** Given 10 failed attempts, When an 11th is made, Then the account locks for 1 hour and returns a generic error (no enumeration).
- **Expired token:** Given a token past its TTL, When an API request uses it, Then verification fails 401 and the client refreshes.
- **Revoked session:** Given an admin revokes a session, When the next refresh occurs (≤ 60 s), Then the session ends and protected requests return 401.
- **Deleted user:** Given a user is deleted, When any of their sessions is used, Then it is rejected and a `user.deleted` webhook fires.
- **SCIM deprovision:** Given an IdP deactivates a user, When the SCIM request is received, Then the user is deactivated and all sessions revoked immediately.
- **Rate limit:** Given >5 sign-in creates in 10s from one IP, When a 6th arrives, Then 429 with `Retry-After` is returned.
- **Org membership change:** Given a user's role changes, When their token next refreshes, Then `o rol / o per` reflect the new role.

PART 28 — API Specification (representative)

Conventions [R]: REST/JSON; Bearer secret key for admin; cursor pagination (`limit` , `offset / cursor`); filtering via query params; `Idempotency-Key` on POST; 429 + `Retry-After` ; errors `{errors:[{code,message,long_message,meta}]}` (Clerk-style).

```

POST /v1/users
Authorization: Bearer sk_live_...
{ "email_address":["a@b.com"], "password":"...", "first_name":"A" }
→ 200 { "id":"user_123","object":"user","email_addresses":[...] }

POST /v1/sessions/{id}/verify → { "status":"active" | "revoked" }
POST /v1/organizations {name, created_by} → org object
POST /v1/organizations/{id}/invitations {email_address, role} →
invitation
GET /v1/jwks → { keys:[{kty,use,kid,alg,n,e}] } # no rate limit

```

FAPI (public, PK-scoped): POST /v1/client/sign_ins, .../attempt_first_factor,
 POST /v1/client/sessions/{id}/tokens .

PART 29 — SDK Design

```

// Client
const { signIn } = useSignIn()
await signIn.create({ identifier: email, password })
const { getToken, userId, isSignedIn } = useAuth()
const token = await getToken() // or getToken({ template:
'supabase' })
const { user } = useUser(); await user.update({ firstName:'A' })

// Server
const { userId, orgId, has } = await auth()
if (!has({ permission:'org:invoices:manage' })) throw forbidden()
const client = createClient({ secretKey })
await client.users.getUser(userId)
await client.organizations.createOrganization({ name, createdBy:userId
})
await client.organizations.createInvitation({ organizationId,
emailAddress, role:'org:member' })

```

Conventions [R]: verb-first async methods returning typed resources; typed
 ClerkAPIError ; strict server/client boundary (separate import paths); full TypeScript
 types; framework adapters wrap a core.

PART 30 — STRIDE Threat Model (condensed)

Component	STRIDE	Mitigation	Detection/Recovery
Auth	Spoofing	MFA, passkeys, breach check	anomalous-login alerts
Sessions	Tampering/Elevation	signed JWT, short TTL, revocation	audit log, revoke

Component	STRIDE	Mitigation	Detection/Recovery
Tokens	Spoofing	RS256, alg allowlist, JWKS	rotate keys
OAuth/SSO	Spoofing	PKCE, state, verified email, nOAuth fix	connection logs
Orgs	Elevation	server-side permission checks	audit trail
APIs	DoS	rate limits, WAF	429 metrics, scale
Webhooks	Tampering	HMAC-SHA256, timestamp	signature-fail alerts
Dashboard	Elevation	RBAC, admin MFA	session recording
DB	Info disclosure	encryption at rest, tenant scoping, KMS	access logs
Internal svcs	Spoofing	mTLS, least privilege	trace anomalies

PART 31 — Implementation Roadmap

- **Phase 0 — Foundations (wks 1–4):** infra (Postgres/Redis/queue/KMS), tenant model, key management, CI/CD, secrets, audit skeleton. *Exit:* keys rotate; JWKS served.
- **Phase 1 — Core identity (wks 5–8):** user CRUD, email verification, metadata, BAPI. *Exit:* full user CRUD.
- **Phase 2 — Authentication (wks 9–12):** password (bcrypt+breach), email codes, lockout, CAPTCHA, rate limits. *Exit:* sign-up/sign-in end to end.
- **Phase 3 — Sessions/tokens (wks 13–16):** hybrid model, client+session tokens, refresh, verification SDK. *Exit:* networkless verify + instant revoke.
- **Phase 4 — OAuth/social + first components (wks 17–20):** Google/GitHub, `<SignIn/>`, JS/Next.js SDK. *Exit:* social login + drop-in UI.
- **Phase 5 — Orgs/RBAC (wks 21–26):** orgs, memberships, roles, invitations, `o` claim, org components. *Exit:* B2B multi-tenant.
- **Phase 6 — Webhooks/dev platform (wks 27–30):** Svix, event catalog, dashboard, JWT templates.
- **Phase 7 — Enterprise (mo 8–12):** SAML/OIDC/EASIE, SCIM, JIT, domain routing.
- **Phase 8 — Scale/reliability (ongoing):** multi-region, partitioning, risk ML, EU residency.

PART 32 — Team Requirements

Role	MVP	V1	Enterprise
Backend	2	4	6
Frontend/components	1	2	3
Security eng	1 (shared)	1	2
Infra/DevOps/SRE	1	2	3
Mobile	0	1	2
DevRel/docs	0.5	1	2
Designer	0.5	1	1
PM	1	1	2
QA/security testing	0.5	1	2
Total	~7	~14	~23

PART 33 — Cost Estimation (rough monthly USD, self-run) [R, estimates]

MAU	DB	Compute	Redis	Queue	Email	SMS	Storage/CDN	Observ
1K	\$50	\$100	\$30	\$10	\$10	\$20	\$20	\$50
10K	\$200	\$300	\$80	\$30	\$50	\$150	\$60	\$150
100K	\$1.2K	\$1.5K	\$400	\$150	\$300	\$1.2K	\$300	\$600
1M	\$8K	\$10K	\$2.5K	\$1K	\$2K	\$8K	\$2K	\$3K
10M	\$50K	\$70K	\$15K	\$8K	\$12K	\$50K	\$12K	\$15K

Assumptions: managed cloud, ~20% of MAU send SMS, token-refresh compute dominates at scale. SMS is the most volatile line.

PART 34 — Master Feature Matrix (abbreviated; every domain represented)

Domain	Feature	User value	Dev value	API	SDK
Auth	Password	familiar	drop-in	FAPI sign_ins	✓

Domain	Feature	User value	Dev value	API	SDK
Auth	Passwordless codes	no password	easy	FAPI attempt	✓
Auth	Social OAuth	1-click	config-only	FAPI oauth	✓
Auth	Passkeys	phishing-resistant	lib	✓	✓
Auth	MFA/TOTP	security	toggle	✓	✓
Session	Hybrid sessions	stay signed in	networkless verify	FAPI tokens	✓
Token	JWT templates	integrations	custom claims	getToken	✓
User	Metadata	personalization	flexible	PATCH metadata	✓
User	Impersonation	support	debug	actor token	✓
Org	Orgs/roles	teams	multi-tenant	✓	✓
Enterprise	SAML/EASIE	corp login	bundled	saml_connections	✓
Enterprise	SCIM	auto-provision	free w/SSO	SCIM 2.0	—
Platform	Webhooks	sync	Svix	✓	verify
Frontend	Components	polished UI	zero-CSS	—	✓
Security	Bot/lockout	safety	built-in	—	✓

PART 35 — Gap Analysis

- **Clerk does extremely well:** DX, prebuilt a11y-optimized components (months of work), the hybrid session model, bundled B2B, EASIE, transparent pricing.
- **Hard to replicate:** the session/token engine's correctness; component polish/a11y; the ML bot/risk engine; SCIM/SAML robustness across IdPs.
- **Needs specialized security expertise:** password hashing/KMS, JWT signing/rotation, OAuth/SAML security, WebAuthn, tenant isolation.
- **Relatively easy:** CRUD APIs, metadata, basic dashboards, webhook consumption.
- **Do NOT build from scratch:** email/SMS delivery, CAPTCHA, webhook infra (Svix), SAML parsing, WebAuthn crypto, KMS.

- **Where a competitor differentiates:** price at scale, self-hosting, EU/data residency, enterprise-first ergonomics, deeper audit/SIEM.
- **Exclude from MVP:** SCIM, SAML, passkeys, multi-region, ML risk, billing product, mobile SDKs.

PART 36 — Final Architectural Recommendation

Layer	Recommendation [R]	Alternatives	Why / tradeoff
Backend lang	Go (auth/session/token) + TypeScript (SDKs/dashboard)	Rust, Java	Go performance+concurrency for token verify; TS for DX parity
Frontend	React + Next.js components; framework adapters	Vue/Svelte	React market gravity
Database	PostgreSQL (Aurora/Cloud SQL)	CockroachDB (multi-region)	relational integrity; JSONB metadata
Cache	Redis	Memcached	data structures for rate limiting/revocation
Queue	Kafka (SQS at small scale)	RabbitMQ	webhook fan-out, audit stream
Search	Postgres FTS → OpenSearch at scale	Algolia	user search
Infra	Kubernetes (EKS) + edge functions for verify	ECS, serverless	control + edge latency
Cloud	AWS primary, multi- region ready	GCP	breadth; KMS/RDS maturity
Email	Resend/Postmark	SES	deliverability + DX
SMS	Twilio	Vonage	reach
Secrets/KMS	AWS KMS + Vault	GCP KMS	signing-key custody
Observability	OTel + Datadog + Grafana	New Relic	cross-service traces
CI/CD	GitHub Actions + Argo CD	GitLab	GitOps

Layer	Recommendation [R]	Alternatives	Why / tradeoff
CAPTCHA/Webhooks	Cloudflare Turnstile + Svix	hCaptcha / self-built	proven, low-liability

Expected scaling: Go token services verify networklessly at the edge (p99 <10ms); Postgres partitioned by tenant; Redis for revocation; Kafka decouples async. Split the modular monolith into services only when a subsystem's scaling profile diverges.

Recommendations

1. **Start now on the foundation, not features.** Phase 0 (KMS, tenant model, key rotation, JWKS, CI/CD, audit) must precede any auth code. A wrong session/token design is expensive to unwind. **Benchmark to proceed:** signing keys rotate with zero downtime and JWKS serves from CDN.
2. **Ship an MVP that mirrors Clerk's day-one DX** — email/password + email-code passwordless + Google/GitHub social, the hybrid session engine, user CRUD + metadata, one polished `<SignIn/>` component, and a Next.js SDK. **Benchmark:** a developer integrates working auth in under 15 minutes from a clean project.
3. **Buy the commodity/high-liability subsystems** (email, SMS/Twilio, CAPTCHA/Turnstile, webhooks/Svix, cloud KMS). Only build the differentiators. This is exactly what Clerk itself does.
4. **Sequence enterprise last** (SAML → OIDC/EASIE → SCIM), but design the Organizations/RBAC data model early so SSO/SCIM slot in cleanly. **Benchmark to start enterprise:** ≥3 paying B2B customers requesting SSO.
5. **Differentiate deliberately on Clerk's gaps:** publish transparent pricing that's cheaper at scale, offer a self-hosting option, and ship EU data residency. These convert the three loudest community complaints into wedges.
6. **Treat the token-refresh storm as a first-class scaling risk.** Instrument refresh QPS from day one and design adaptive refresh + configurable TTL before you approach 1M users.
7. **Staffing thresholds:** hire a dedicated security engineer before writing password/JWT/OAuth code; add mobile and DevRel only at V1; scale infra/SRE headcount when you cross ~100K MAU or add multi-region.

Caveats

- **The full webhook event catalog is Dashboard-only** in Clerk's docs; the enumerated event names here come from Clerk's official agent "skills" and corroborating

integration docs, labeled [V/I]. Verify against a live instance before depending on exact names. **No dedicated SAML/SCIM webhook event types were found** [U].

- **RS256 as the signing algorithm** is inferred from Clerk's public/private-key language and JWT header conventions; Clerk documents the mechanism (private-key sign, public-key/JWKS verify) but the exact algorithm identifier is not stated verbatim in the pages reviewed [V/I].
- **Pricing is a live target.** All pricing is verified as of the February 5, 2026 change to 50,000 MRU free and cross-checked in mid-2026; some third-party pages still cite the retired 10,000-MAU tier. Competitor pricing (WorkOS, Auth0, Supabase, Firebase) is verified from 2026 sources but changes frequently.
- **Cost, staffing, timeline, schema, and stack figures are recommendations/estimates [R]**, not Clerk internals (Clerk's actual infrastructure is not public). Treat the cost table as directional; SMS and compute dominate variance.
- **"Instant" revocation is bounded by the ~60s token lifetime** — a revoked session's already-issued token remains valid until its next refresh fails ($\leq 60s$), except for SCIM deprovisioning which Clerk documents as immediate session revocation.
- Community-sourced DX complaints reflect developer sentiment, not Clerk defects, and some (e.g., lock-in) are partially mitigated by features like self-serve export.